Apuntes del Sprint 4 · Python para datos: de la hoja de cálculo al programa

Estos apuntes **explican paso a paso qué hace cada línea**, con ejemplos pequeños cuyo resultado puedes comprobar con una calculadora. Úsalos **durante las misiones** cuando algo se te atasque, **en casa para repasar** o cuando algo te salga en rojo. No hace falta haber programado nunca. **Programar es dar instrucciones muy precisas y en orden**, como una receta: el ordenador hace exactamente lo que escribes, ni más ni menos.

Índice

- Cómo estudiar estos apuntes
- 0. De qué va el sprint (en 1 minuto)
- **Día 1 · Martes**: la lógica, con los datos en memoria (misiones 0 a 3)
  - 1. Dónde se ejecuta Python
  - 2. Variables, tipos y operadores
  - 3. Decidir y repetir: if, for, while
  - 4. Funciones: encapsular un cálculo
  - 5. Listas, tuplas, diccionarios y conjuntos
  - 6. Una lista de diccionarios es una tabla
- **Día 2 · Miércoles**: ficheros, errores y entrega (misiones 4 y 5)
  - 7. Leer CSV y guardar JSON
  - 8. Cuando algo falla: excepciones
  - 9. El reto, función a función
- **Para casa**: material extra
  - 10. Entorno limpio: venv, pip y Git en el terminal
  - 11. Buenas prácticas y POO
- **Para consultar**
  - 12. Dónde practicar cada cosa
  - 13. Chuleta de vocabulario

Cómo estudiar estos apuntes

1. **Antes de cada misión** (o de cada cuaderno, si repasas en casa), lee la sección correspondiente: la tabla de la sección 12 te dice cuál.
2. **Escribe el código tú, no lo copies**: los fallos que tengas enseñan más que acertar a la primera.
3. **Cambia números y predice** qué saldrá antes de ejecutar.
4. **Cuando salga un error en rojo, lee la última línea**: casi siempre dice qué pasó (consulta la tabla de errores de la sección 8).
5. **En el reto, no empieces por el final:** pruebas en rojo → una función → verde → commit.

---

0. De qué va el sprint (en 1 minuto)

TurisData calcula sus cifras **a mano en una hoja de cálculo**. Esa hoja se rompe, nadie sabe quién tocó qué y no se puede repetir. Python permite convertir esos cálculos en **un programa que cualquiera ejecuta y comprueba**.

Pieza	Para qué sirve	Cuaderno (en casa)
Variables, tipos, `if`, `for`	Guardar datos y tomar decisiones	S04_01
Funciones	Encapsular un cálculo para reutilizarlo	S04_02
Listas, diccionarios, conjuntos	Representar los datos reales	S04_03
CSV, JSON y errores	Leer y guardar ficheros sin que el programa «explote»	S04_04
Clases y buenas prácticas (ampliación)	Organizar código grande	S04_05

El reto: `kpis.py`, un script que **lee el CSV de reservas, calcula 3 KPIs, avisa si el fichero está mal y guarda un JSON**, con pruebas que lo comprueban.

DÍA 1 · Martes · La lógica, con los datos en memoria

Misiones 0 a 3 del reto: variables y tipos, listas y diccionarios, decidir y repetir, funciones con pruebas.

1. Dónde se ejecuta Python

Dónde	Cuándo	Ojo con
Google Colab	En el aula: no hay que instalar nada	Al cerrar la sesión **se pierde el estado**
Jupyter / VS Code	En tu equipo, con tus ficheros	Hay que instalar Python y las librerías
Terminal (`python kpis.py`)	Para el script del reto	Es lo que usará TurisData

Un **cuaderno** (notebook) mezcla texto y **celdas de código** que se ejecutan una a una. Eso es cómodo, pero tiene una trampa: las celdas recuerdan lo que ejecutaste antes **en el orden en que lo hiciste**, no en el orden en que aparecen.

> **Regla de oro del cuaderno:** si algo se comporta raro, **Reiniciar entorno y ejecutar todo de arriba abajo**. El 80 % de los fallos misteriosos se arreglan así.

2. Variables, tipos y operadores

Una **variable** es una **caja con nombre** donde guardas un valor.

```
noches, precio_noche, tasa = 5, 72.50, 1.50
total_estancia = noches * (precio_noche + tasa)
print(f"{noches} noches: {total_estancia:.2f} EUR")
```

Resultado: `5 noches: 370.00 EUR` (porque 72,50 + 1,50 = 74 y 5 × 74 = 370).

- `=` **asigna** (mete un valor en la caja). No es el igual de matemáticas.
- `print(...)` muestra algo en pantalla.
- `f"…{variable}…"` es un **f-string**: pone el valor de la variable dentro del texto. `:.2f` significa «con 2 decimales».

Los cuatro tipos básicos

Tipo	Qué es	Ejemplo
`int`	Número entero	`5`
`float`	Número con decimales	`72.50` (con **punto**, no coma)
`str`	Texto	`"directo"`
`bool`	Verdadero o falso	`True`, `False`

Operadores

Operador	Qué hace	Ejemplo
`+ - *`	Suma, resta, multiplica	`3 * 4` → `12`
`/`	División normal	`7 / 2` → `3.5`
`//`	División **entera** (sin decimales)	`7 // 2` → `3`
`%`	**Resto** de la división	`7 % 2` → `1`
`==`	¿Son iguales?	`canal == "web"`
`and`, `or`, `not`	Combinar condiciones	`a and b`

Error clásico: `==` **compara**, `=` **asigna**. `if canal = "web":` da error.

Limpiar texto

Los datos reales vienen sucios. Los métodos de texto se **encadenan**:

```
canal_bruto = "  Web Intermediaria "
canal = canal_bruto.strip().lower().replace(" ", "_")
# strip() quita espacios de los extremos → "Web Intermediaria"
# lower() pasa a minúsculas              → "web intermediaria"
# replace(" ", "_") cambia espacios      → "web_intermediaria"
```

> **Si vienes de Java:** no hay `;` ni llaves. Los bloques se marcan con **sangría de 4 espacios** y no declaras el tipo: lo lleva el valor.

3. Decidir y repetir: if, for, while

if: decidir

```
if tarifa == "flexible" and deposito == "sin_deposito":
    riesgo = "alto"
elif tarifa == "flexible" or deposito == "sin_deposito":
    riesgo = "medio"
else:
    riesgo = "bajo"
```

Se lee: *si las dos cosas se cumplen → alto; si no, si se cumple al menos una → medio; en cualquier otro caso → bajo*. Python evalúa **de arriba abajo y se queda con el primer caso que se cumple**. Los dos puntos `:` y la **sangría** son obligatorios.

for: repetir recorriendo una secuencia

Una estancia de 3 noches, con precio base 60 € que sube 5 € cada noche:

```
total, precio = 0, 60
for noche in range(3):          # noche vale 0, luego 1, luego 2
    total += precio + 5 * noche
```

Paso a paso:

Vuelta	noche	Se suma	total
1	0	60 + 0 = 60	60
2	1	60 + 5 = 65	125
3	2	60 + 10 = 70	**195**

`total += x` es lo mismo que `total = total + x`. `range(3)` genera **0, 1, 2** (empieza en 0 y **no incluye** el 3).

while: repetir mientras se cumpla algo

```
intentos = 0
while intentos < 3:
    intentos += 1
```

> **Dentro de un `while` algo tiene que cambiar**, o se repite para siempre (bucle infinito). Si se te cuelga el cuaderno, pulsa *Interrumpir* y revisa eso.

4. Funciones: encapsular un cálculo

Una **función** es una **receta con nombre**: le das ingredientes (parámetros) y te devuelve un resultado (`return`).

```
def precio_final(noches, precio_noche, descuento=0.0):
    return round(noches * precio_noche * (1 - descuento), 2)
```

Llamada	Cálculo	Resultado
`precio_final(4, 80)`	4 × 80 × (1 − 0)	`320.0`
`precio_final(10, 80, 0.1)`	10 × 80 × (1 − 0,1) = 800 × 0,9	`720.0`

- `descuento=0.0` es un **parámetro por defecto**: si no lo das, vale 0.
- `return` **devuelve** el valor; `print` solo lo muestra. Una función que hace `print` pero no `return` **no se puede reutilizar** en otros cálculos.

```
def clasificar_antelacion(dias):
    """Devuelve 'ultima_hora', 'anticipada' o 'muy_anticipada'."""
    if dias < 30:
        return "ultima_hora"
    return "anticipada" if dias < 90 else "muy_anticipada"
```

El texto entre triples comillas es el **docstring**: explica qué hace la función. Con ella: 10 días → `ultima_hora`; 30 y 89 → `anticipada`; 90 → `muy_anticipada`.

Otras piezas del cuaderno S04_02:

- `*args`: permite pasar **cualquier número** de valores (`maximo_ingreso(200, 450.5)`).

- `lambda`: una función de una sola línea sin nombre.
- **Módulos:** código ya hecho que se importa (`import datetime`, `import statistics`).

Probar con assert

Cada función lleva **una prueba** que comprueba que hace lo que debe:

```
assert precio_final(4, 80) == 320.0
assert clasificar_antelacion(10) == "ultima_hora"
```

Si es cierto, no pasa nada (**silencio = bien**). Si es falso, salta un `AssertionError` que te avisa del fallo. Eso es una «prueba».

5. Listas, tuplas, diccionarios y conjuntos

Estructura	Se escribe	Rasgo clave	Ejemplo
Lista	`[ ]`	Ordenada y **modificable**	`[3, 5, 2, 4]`
Tupla	`( )`	Ordenada e **inmutable**	`(28.46, -16.25)` (coordenadas)
Diccionario	`{ }`	**Clave → valor**	`{"canal": "directo", "noches": 3}`
Conjunto	`{ }`	**Sin repetidos**	`{"web", "directo"}`

Lista y cortes:

```
noches_semana = [3, 5, 2, 4, 6, 1, 7]
noches_semana[0]        # 3  (la primera posición es la 0)
noches_semana[-1]       # 7  (el último)
noches_semana[-3:]      # [6, 1, 7]  (los tres últimos)
sum(noches_semana[-3:]) # 14
```

Diccionario: como una ficha. Se accede por la clave: `reserva["noches"]` → `3`.

Conjunto: `len(set(["web", "web", "directo"]))` → `2` (cuenta valores distintos).

Contar con un diccionario (muy usado):

```
conteo = {}
for canal in ["web", "web", "directo"]:
    conteo[canal] = conteo.get(canal, 0) + 1
# {'web': 2, 'directo': 1}
```

`conteo.get(canal, 0)` significa «dame el valor de `canal`, y si no existe todavía, dame 0».

6. Una lista de diccionarios es una tabla

Una **reserva es un diccionario**; un **fichero de reservas, una lista de diccionarios**:

```
reservas = [
    {"canal": "directo", "noches": 3, "precio_noche": 80.0,  "cancelada": 0},
    {"canal": "agencia", "noches": 5, "precio_noche": 95.0,  "cancelada": 1},
    {"canal": "agencia", "noches": 2, "precio_noche": 100.0, "cancelada": 0},
    {"canal": "directo", "noches": 4, "precio_noche": 90.0,  "cancelada": 0},
]
```

Cada diccionario es **una fila**; cada clave es **una columna**. Igual que en una hoja de cálculo.

Comprensión de lista (un bucle en una línea)

Ingresos de las reservas **no canceladas**:

```
ingresos_confirmados = [r["noches"] * r["precio_noche"]
                        for r in reservas if r["cancelada"] == 0]
```

Se lee: *«para cada reserva r, si no está cancelada, calcula noches × precio»*. Resultado: `[240.0, 200.0, 360.0]` (3×80, 2×100, 4×90). La reserva cancelada de agencia se queda fuera.

Tasa de cancelación por canal

Se **acumulan dos contadores** en dos diccionarios (total y canceladas):

```
total, canceladas = {}, {}
for r in reservas:
    c = r["canal"]
    total[c] = total.get(c, 0) + 1
    canceladas[c] = canceladas.get(c, 0) + r["cancelada"]
tasa = {c: round(100 * canceladas[c] / total[c], 1) for c in total}
```

Resultado: `{'directo': 0.0, 'agencia': 50.0}`: de las 2 reservas de agencia, 1 se canceló → 50 %.

DÍA 2 · Miércoles · Ficheros, errores y entrega

Misiones 4 y 5 del reto: leer el CSV sin que se rompa, guardar el JSON y entregar.

7. Leer CSV y guardar JSON

Un **CSV** es una tabla en texto plano (valores separados por comas). Un **JSON** es un diccionario/lista en texto: es el formato que usan casi todas las aplicaciones para intercambiar datos.

```
import csv, json

with open("reservas_turisdata.csv", newline="", encoding="utf8") as f:
    filas = list(csv.DictReader(f))        # lista de diccionarios

noches = int(filas[0]["noches"])           # hay que CONVERTIR

with open("kpis.json", "w", encoding="utf8") as f:
    json.dump(kpis, f, ensure_ascii=False, indent=2)   # guarda con tildes y sangría
```

Puntos importantes:

- **Todo llega como TEXTO.** Aunque en el CSV ponga `3`, Python lo lee como `"3"`. Para sumar hay que convertir: `int("3")` → `3`; `float("72.5")` → `72.5`. Si no, `"3" + "4"` da `"34"` (texto pegado), no 7.
- `csv.DictReader` te da **cada fila como diccionario**.
- `with open(...) as f:` **cierra el archivo solo** al terminar el bloque.
- `encoding="utf8"` evita que las tildes salgan como símbolos raros; `ensure_ascii=False` las conserva al escribir el JSON; `indent=2` lo deja legible.

8. Cuando algo falla: excepciones

Un programa real **se encuentra datos malos**: un campo vacío, un fichero que no existe. Si no lo controlas, el programa **explota**. Con `try/except` lo **gestionas**:

```
def a_float_seguro(texto, defecto=None):
    try:
        return float(texto)
    except (ValueError, TypeError):
        return defecto
```

Llamada	Resultado
`a_float_seguro("12.5")`	`12.5`
`a_float_seguro("abc")`	`None` (no se pudo convertir)
`a_float_seguro(None, 0.0)`	`0.0`

- Capturamos **un tipo concreto** de error (`ValueError`, `TypeError`, `FileNotFoundError`, `KeyError`). **Nunca un `except:` vacío**: esconde todos los fallos, incluidos los que no esperabas.
- `else:` se ejecuta si **no** hubo error; `finally:` se ejecuta **siempre**.
- `raise FileNotFoundError("No encuentro el fichero: …")` **lanza tú mismo** un error con un mensaje claro.

> **Descartar filas malas sin contarlas es peligroso.** Si el programa tira 300 filas sin avisar, los KPIs estarán mal y nadie lo sabrá. Por eso `leer_reservas` devuelve también **los avisos** y las **filas descartadas**.

Errores frecuentes en Python

Mensaje	Qué significa	Qué mirar
`NameError`	Usas una variable que no existe	¿La ejecutaste antes? ¿Mal escrita?
`SyntaxError`	Falta un `:`, comilla o paréntesis	La línea anterior a la que señala
`IndentationError`	Sangría mal	Usa siempre 4 espacios
`TypeError`	Mezclas tipos (`"3" + 4`)	Convierte con `int()` o `float()`
`KeyError`	Pides una clave que no existe	¿Nombre de columna exacto?
`FileNotFoundError`	No encuentra el fichero	Carpeta y nombre del archivo
`ZeroDivisionError`	Divides entre 0	Comprueba que el total no es 0

9. El reto, función a función

El reto reúne lo que has practicado. Cada función de la plantilla se construye en una misión y tiene un cuaderno de refuerzo:

Función	Qué hace	Se construye en	Refuerzo en casa
`leer_reservas(ruta)`	Lee, valida y devuelve `(reservas, avisos)`	Misión 4	S04_04
`tasa_cancelacion_por_canal`	% de canceladas por canal (1 decimal)	Misión 3	S04_03
`ingreso_medio_por_reserva`	Media de noches × precio de las **no canceladas**	Misión 3	S04_03
`ocupacion_proxy`	Ocupación aproximada con la capacidad	Misión 3 (en pantalla, con el docente)	S04_03
`exportar_json`	Guarda el resultado con tildes y sangría	Misión 5	S04_04

Con el ejemplo del apartado 6, `ingreso_medio_por_reserva` sería (240 + 200 + 360) / 3 = **266,67 €**.

Orden de trabajo en el reto: pruebas primero (desde el martes)

1. **Ejecuta `python test_kpis.py`: tiene que fallar** (todavía no hay código).
2. **Rellena una función.**
3. **Vuelve a ejecutar:** esa prueba **pasa a verde**.
4. **Commit** (guarda una «foto» de lo que ya funciona). Y a la siguiente.

Así siempre sabes **qué funciona y qué no**, y avanzas sin miedo a romper lo anterior.

Un **commit** es una «foto» de tu trabajo con un **mensaje** que dice **qué cambia y por qué**. Mensaje malo: «cambios». Mensaje bueno: «Valida columnas obligatorias en leer_reservas». En el reto, cada subida de ficheros a GitHub desde el navegador es un commit (lo explica «Cómo entregar en GitHub»); hacen falta al menos tres, de personas distintas.

PARA CASA · Material extra

Nada de esto hace falta para terminar el reto: es para quien quiera ir más allá.

10. Entorno limpio: venv, pip y Git en el terminal

```
python -m venv .venv                # crea un entorno aislado para ESTE proyecto
source .venv/bin/activate           # lo activa (Windows: .venv\Scripts\activate)
pip install pandas scikit-learn     # instala librerías solo ahí dentro
git init && git add . && git commit -m "Primer commit"
```

- Un **venv por proyecto** evita el famoso «en mi ordenador funciona».
- `requirements.txt` **anota las versiones** para que otra persona lo reproduzca.
- **Git** guarda «fotos» de tu trabajo (qué es un *commit* y cómo escribir su mensaje: sección 9). Aquí lo haces desde el terminal, en vez de subir los ficheros desde el navegador.

11. Buenas prácticas y POO

- **PEP 8:** `nombres_con_guion_bajo`, 4 espacios, docstring en cada función.
- **Funciones pequeñas:** leer, calcular y exportar son **funciones distintas**, no una gigante.
- **Commits con mensaje útil.**
- **POO** (cuaderno S04_05, ampliación): una **clase** es un molde para crear objetos. `Reserva` agrupa los datos de una reserva y **un método** `ingreso()` que calcula noches × precio. Una clase hija `ReservaGrupo` **hereda** de `Reserva` y le añade un 10 % de descuento. No es necesaria para el reto.

---

12. Dónde practicar cada cosa

Si no entiendes...	Mira la sección	En clase	En casa
Variables, tipos, `if`, `for`, `while`	2-3	Misiones 0 y 2	Cuaderno `S04_01_entorno_sintaxis`
Funciones, `assert`, `lambda`	4	Misión 3	`S04_02_funciones_modulos`
Listas, diccionarios, comprensiones	5-6	Misión 1	`S04_03_estructuras_datos`
CSV, JSON, `try/except`	7-8	Misiones 4 y 5	`S04_04_ficheros_excepciones`
Pruebas en verde, commits, el reto completo	9	Misiones 3 a 5 (tarjetas de misión)	—

Si no entiendes...	Mira la sección	En clase	En casa
Entorno limpio y Git en el terminal	10	—	Material extra
Clases y herencia	11	—	`S04_05_poo_buenas_practicas` (opcional)

13. Chuleta de vocabulario

Palabra	En lengua normal
Variable	Caja con nombre donde guardas un valor
Tipo	Clase de dato: entero, decimal, texto, verdadero/falso
Función	Receta con nombre: entran datos, sale un resultado
Parámetro	Dato que le das a la función
`return`	Devolver el resultado al que llamó
Docstring	Texto que explica qué hace la función
Lista	Colección ordenada y modificable
Diccionario	Colección de pares clave → valor
Conjunto	Colección sin repetidos
Comprensión	Crear una lista con un bucle en una línea
CSV / JSON	Formatos de texto para guardar datos
Excepción	Error que puedes capturar y gestionar
`assert`	Comprobación automática: si es falsa, avisa
Entorno virtual (`venv`)	Instalación de Python aislada por proyecto
Commit	«Foto» guardada de tu trabajo en Git
Módulo	Código ya hecho que se importa